



Cancer Workshop 2026 - Module 8: Pathway Enrichment Analysis

Veronique Voisin

October 2026

Contents

Presentation of the dataset	5
Differential Expression	7
0.1 Install and load libraries	7
0.2 Data used for this comparison	8
0.3 Filter low-count genes and TMM-normalize	9
0.4 Design (~0 + group) and named contrast	10
0.5 Dispersion estimation and quasi-likelihood GLM	11
0.6 Calculating ranking score and export full results	11
0.7 Result output	14
0.8 Version of packages and R	14
Pathway Enrichment Analysis using a defined genelist	17
0.9 Introduction	17
0.10 Goal of the exercise 1	17
0.11 Data	18
0.12 Exercise 1 - prepare the gene lists	18
0.13 Exercise 2 - visualization	29
Ranked gene list	31

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

This module will cover how to use differential expression output to generate pathway enrichment results.

Presentation of the dataset

- This module is a continuation of the transcriptomics module 6 and we will use the same dataset. This dataset contains bulk RNA-seq data of a model of breast cancer. The gene counts were deposited to the Gene Expression Omnibus (GEO) database under accession ***GSE222367*** and are publicly available.
- RNA was extracted from MCF7, a breast cancer cell line under different conditions: parental cell line and cell line samples resistant to the drugs palbociclib or abemaciclib.
- Biological replicates were available for each condition.
- RNA-seq library was prepared and sequenced using an Illumina HiSeq 2500 sequencer.
- Sequence reads were mapped to the human reference genome (Gencode GRCh38) using TopHat as aligner.
- The BAM files with mapped reads for each sample were sorted and then used as input for HTSeq 0.11.0 in order to generate read count data at the gene level.
- We retrieved the gene counts and performed differential expression analysis using a negative binomial generalized linear model, followed by pathway enrichment analysis.



ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE222367
Country USA
Platforms (1) GPL16791 Illumina HiSeq 2500 (Homo sapiens)
Samples (42) Less...
GSM6921637 MCF7 Parental biol rep 1
GSM6921638 MCF7 Parental biol rep 2
GSM6921639 MCF7 Parental biol rep 3
GSM6921640 MCF7 1.2 Palbo-resistant biol rep 1
GSM6921641 MCF7 1.2 Palbo-resistant biol rep 2
GSM6921642 MCF7 1.2 Palbo-resistant biol rep 3

Figure 1: GEO dataset

Differential Expression

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

We perform differential expression analysis using a negative binomial generalized linear model using the edgeR package.

0.1 Install and load libraries

```
## CRAN and Bioconductor setup
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

## List of CRAN packages
cran_packages <- c(
  "tidyverse",
  "gprofiler2",
  "ggrepel"
)

# List of Bioconductor packages
bioc_packages <- c(
  "edgeR",
  "fgsea"
)

# Install CRAN packages if not already installed
for (pkg in cran_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
}
```

```

# Install Bioconductor packages if not already installed
for (pkg in bioc_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    BiocManager::install(pkg)
  }
}

# Load the libraries

# CRAN packages
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.3      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to resolve.

library(gprofiler2)
library(ggrepel)
# Bioconductor packages
library(edgeR)

## Loading required package: limma
library(fgsea)

```

0.2 Data used for this comparison

For this analysis, we are going to compare the Palbo-resistant cell lines (3 biological replicates) versus the parental cell lines (3 biological replicates)

0.2.1 generate a DGE object

```

# 1. Find the count files
files <- list.files(path = "data", pattern = "\\..txt\\.gz$", full.names = TRUE)

# 2. Sample names from the file names (strip the extension)
samples <- sub("_S.*", "", basename(files))

```

Country	USA
Platforms (1)	GPL16791 Illumina HiSeq 2500 (Homo sapiens)
Samples (42)	GSM6921637 MCF7 Parental biol rep 1 GSM6921638 MCF7 Parental biol rep 2 GSM6921639 MCF7 Parental biol rep 3 GSM6921640 MCF7 1.2 Palbo-resistant biol rep 1 GSM6921641 MCF7 1.2 Palbo-resistant biol rep 2 GSM6921642 MCF7 1.2 Palbo-resistant biol rep 3

Figure 2: GEO dataset

```
# 3. Read the files into a DGEList
# HTSeq output has no header: column 1 = gene ID, column 2 = count.
# R reads gzipped files directly, so no need to decompress.
dge <- readDGE(files, columns = c(1, 2), header = FALSE, labels = samples)
```

```
## Meta tags detected: __no_feature, __ambiguous, __too_low_aQual, __not_aligned, __alignment_not
```

```
# 4. Remove HTSeq summary rows (__no_feature, __ambiguous, etc.)
dge <- dge[!grepl("^__", rownames(dge)), , keep.lib.sizes = FALSE]
```

```
# 5. (Optional) add group information
dge$samples$group <- factor(
  c("parental", "parental", "parental",
    "palbo_resistant", "palbo_resistant", "palbo_resistant"),
  levels = c("parental", "palbo_resistant") # parental = reference
)
```

0.3 Filter low-count genes and TMM-normalize

```
keep <- filterByExpr(dge, group = dge$samples$group)
dge <- dge[keep, , keep.lib.sizes = FALSE]
dge <- calcNormFactors(dge, method = "TMM")
cat("Genes retained after filtering:", nrow(dge), "\n")
```

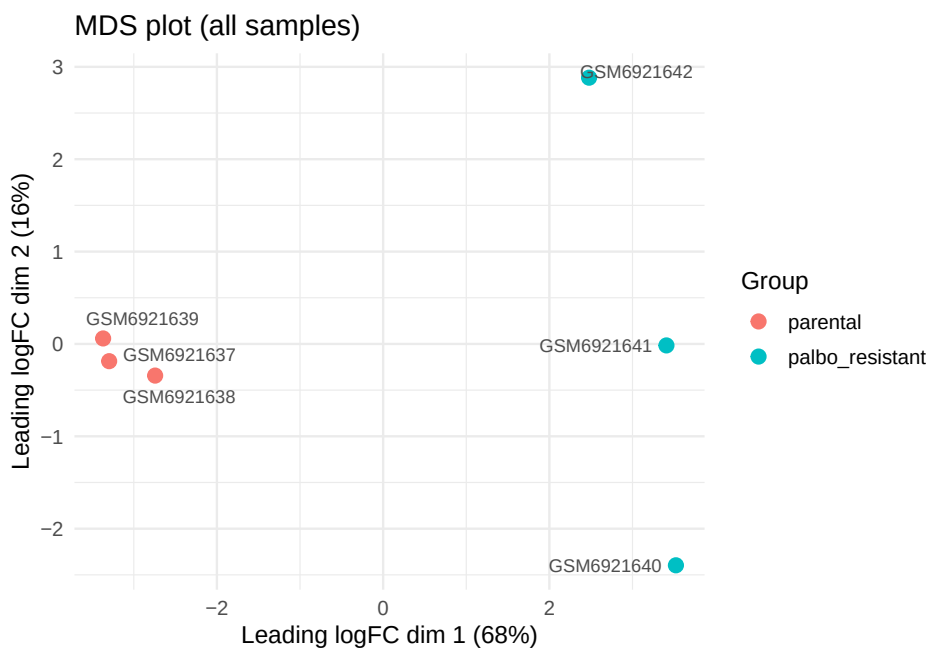
```
## Genes retained after filtering: 15246
```

```
mds <- plotMDS(dge, col = as.numeric(dge$samples$group), main = "MDS (logCPM)")
```

```
group = dge$samples$group
var_pct <- round(100 * mds$var.explained[mds$dim.plot])
mds_df <- data.frame(dim1 = mds$x, dim2 = mds$y,
  sample = colnames(dge), group = group)

ggplot(mds_df, aes(x = dim1, y = dim2, color = group)) +
```

```
geom_point(size = 3) +
ggrepel::geom_text_repel(aes(label = sample), size = 3, color = "grey30",
                          show.legend = FALSE, max.overlaps = Inf,
                          box.padding = 0.4, segment.color = "grey60") +
labs(title = "MDS plot (all samples)",
     x = sprintf("Leading logFC dim %d (%d%%)", mds$dim.plot[1], var_pct[1]),
     y = sprintf("Leading logFC dim %d (%d%%)", mds$dim.plot[2], var_pct[2]),
     color = "Group") +
theme_minimal(base_size = 12) +
theme(legend.position = "right")
```



0.4 Design ($\sim 0 + \text{group}$) and named contrast

```
design <- model.matrix(~ 0 + group, data = dge$samples)
colnames(design) <- levels(dge$samples$group) # "parental", "palbo_resistant"

contr <- makeContrasts(
  resistant_vs_parental = palbo_resistant - parental, # + logFC = up in resistant
  levels = design
)

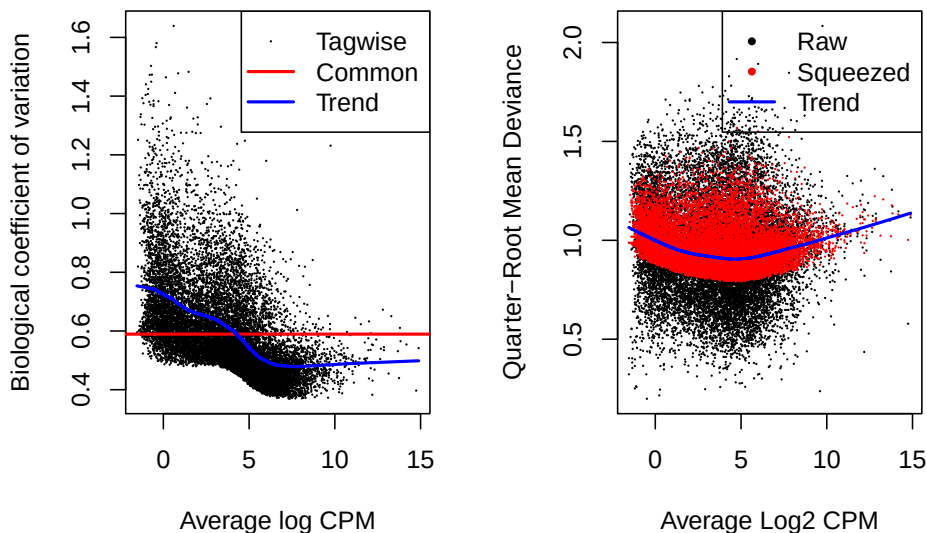
head(design)
```

```
##          parental palbo_resistant
## GSM6921637         1             0
## GSM6921638         1             0
## GSM6921639         1             0
## GSM6921640         0             1
## GSM6921641         0             1
## GSM6921642         0             1
```

0.5 Dispersion estimation and quasi-likelihood GLM

```
dge <- estimateDisp(dge, design, robust = TRUE)
fit <- glmQLFit(dge, design, robust = TRUE)

par(mfrow = c(1, 2)); plotBCV(dge); plotQLDisp(fit)
```



```
qlf <- glmQLFTest(fit, contrast = contr[, "resistant_vs_parental"])
res <- topTags(qlf, n = Inf, sort.by = "PValue")$table
res$gene <- rownames(res)
```

0.6 Calculating ranking score and export full results

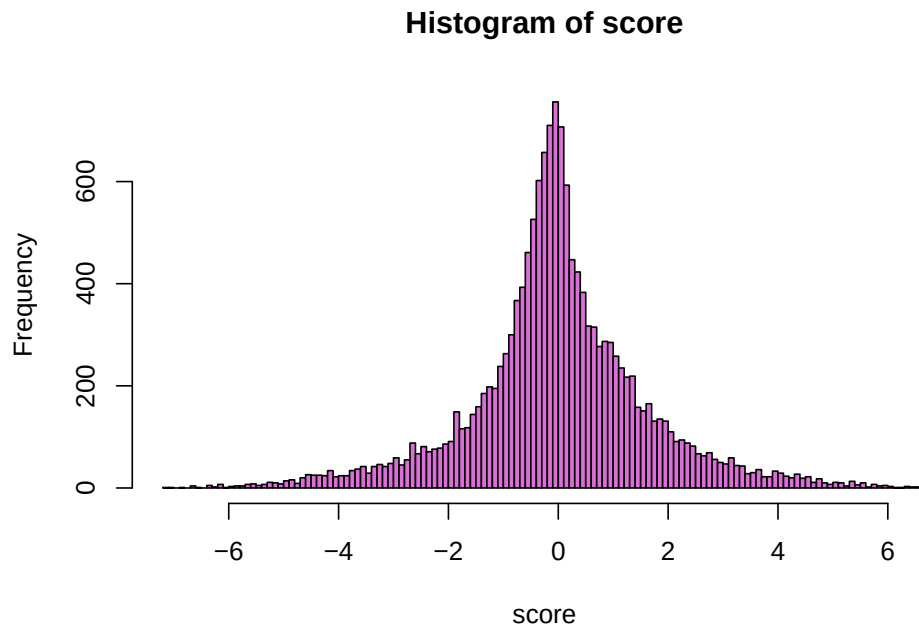
```
res$score = sign(res$logFC) * -log10(res$PValue)
head(res)
```

```
##          logFC  logCPM      F      PValue      FDR  gene      score
## FOXA1   -7.733381 7.036614 212.9072 7.319997e-08 0.0003543738 FOXA1 -7.135489
## ESRP1  -12.579669 6.455576 208.4055 8.469988e-08 0.0003543738 ESRP1 -7.072117
## AZGP1  -11.099954 7.258554 184.8560 1.464633e-07 0.0003543738 AZGP1 -6.834271
## EPN3   -11.064332 6.286707 169.9951 2.143977e-07 0.0003543738 EPN3  -6.668780
## AP1M2  -12.059269 6.763487 169.5441 2.169978e-07 0.0003543738 AP1M2 -6.663545
## CAV1   10.118680 6.613522 165.7532 2.404307e-07 0.0003543738 CAV1   6.619010
```

```
## visualization
```

```
### histogram of the ranking score
```

```
hist(res$score, breaks=100, col="orchid", xlab = "score", main = "Histogram of score")
```



```
### volcano plot
```

```
# Step 1: add -log10(FDR), and label each gene Up / Down / Not sig
res <- res %>%
```

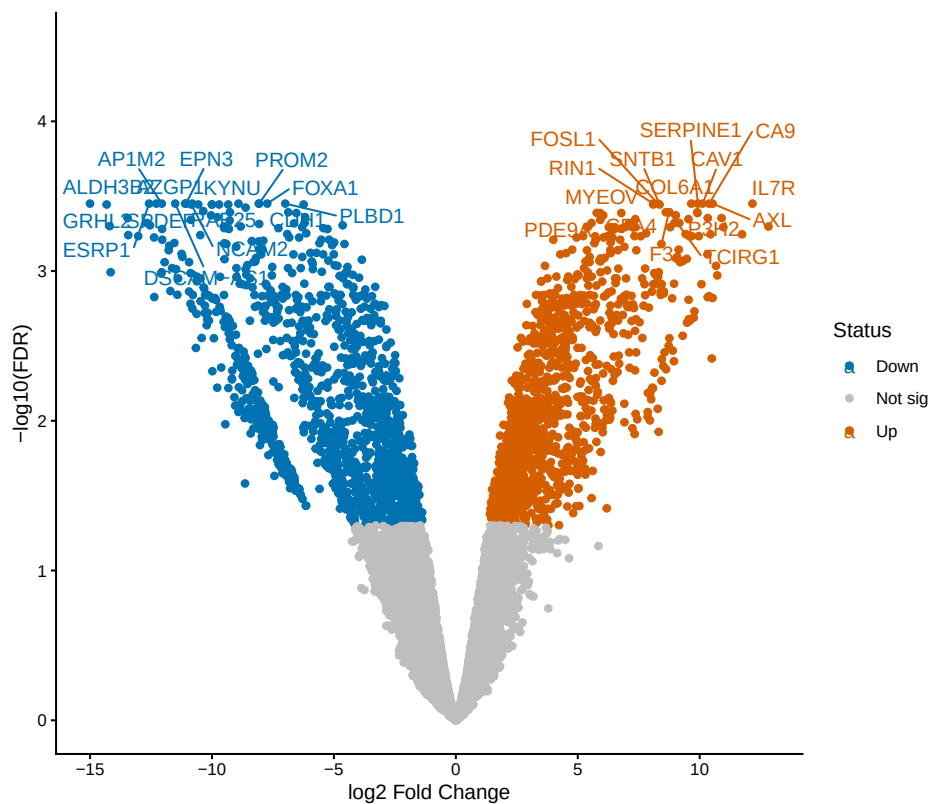
```
  mutate(
    negLogFDR = -log10(FDR),
    status = case_when(
      FDR < 0.05 & logFC > 0 ~ "Up",
      FDR < 0.05 & logFC < 0 ~ "Down",
      TRUE ~ "Not sig"
    )
  )
```

0.6. CALCULATING RANKING SCORE AND EXPORT FULL RESULTS13

```
)

# Step 2: pick the top 15 most significant genes on each side, to label
top_up   <- res %>% filter(status == "Up")   %>% arrange(FDR) %>% slice_head(n = 15)
top_down <- res %>% filter(status == "Down") %>% arrange(FDR) %>% slice_head(n = 15)
top_genes <- bind_rows(top_up, top_down)

# Step 3: plot
ggplot(res, aes(x = logFC, y = negLogFDR, color = status)) +
  geom_point() +
  scale_color_manual(values = c("Up" = "#D55E00", "Down" = "#0072B2", "Not sig" = "grey")) +
  ggrepel::geom_text_repel(data = top_genes, aes(label = gene), max.overlaps = Inf) +
  labs(x = "log2 Fold Change", y = "-log10(FDR)", color = "Status") +
  coord_cartesian(ylim = c(0, 4.5)) +
  theme_classic()
```



0.7 Result output

```
print(summary(decideTests(qlf, adjust.method = "BH", p.value = 0.05)))

##          -1*parental 1*palbo_resistant
## Down                                1325
## NotSig                             12579
## Up                                  1342

write.csv(res, "results/edgeR_resistant_vs_parental_all_genes.csv", row.names = FALSE)
```

0.8 Version of packages and R

```
sessionInfo()

## R version 4.3.2 (2023-10-31)
## Platform: aarch64-apple-darwin20 (64-bit)
## Running under: macOS 15.5
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Toronto
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices datasets  utils      methods    base
##
## other attached packages:
## [1] fgsea_1.28.0    edgeR_4.0.16    limma_3.58.1    ggrepel_0.9.8
## [5] gprofiler2_0.2.4 lubridate_1.9.5 forcats_1.0.1    stringr_1.6.0
## [9] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
## [13] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] fastmatch_1.1-8    gtable_0.3.6      xfun_0.61
## [4] htmlwidgets_1.6.4  lattice_0.22-7     tzdb_0.5.0
## [7] vctrs_0.7.3        tools_4.3.2        generics_0.1.4
## [10] parallel_4.3.2     pkgconfig_2.0.3    Matrix_1.6-5
## [13] data.table_1.18.6.1 RColorBrewer_1.1-3 S7_0.2.2
## [16] lifecycle_1.0.5    compiler_4.3.2     farver_2.1.2
```

```
## [19] statmod_1.5.2      tinytex_0.61      codetools_0.2-20
## [22] htmltools_0.5.9    yaml_2.3.12       plotly_4.12.1
## [25] pillar_1.11.1      BiocParallel_1.36.0 tidyselect_1.2.1
## [28] locfit_1.5-9.12    digest_0.6.39     stringi_1.8.9
## [31] bookdown_0.48      splines_4.3.2     labeling_0.4.3
## [34] cowplot_1.2.0      fastmap_1.2.0     grid_4.3.2
## [37] cli_3.6.6          magrittr_2.0.5    withr_3.0.3
## [40] scales_1.4.0       timechange_0.4.0  rmarkdown_2.32
## [43] httr_1.4.9         otl_0.2.0         hms_1.1.4
## [46] evaluate_1.0.5     knitr_1.52        viridisLite_0.4.3
## [49] rlang_1.3.0        Rcpp_1.1.2        glue_1.8.1
## [52] BiocManager_1.30.27 renv_1.2.4         rstudioapi_0.19.0
## [55] jsonlite_2.0.0     R6_2.6.1
```


Pathway Enrichment Analysis using a defined genelist

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

Authors: Veronique Voisin, Ruth Isserlin

Presenter: Veronique Voisin

0.9 Introduction

During this practical lab, we will perform pathway enrichment analysis using a defined gene list.

0.10 Goal of the exercise 1

For this exercise, our goal is to run pathway enrichment analysis using g:Profiler and explore the results. g:Profiler performs a gene-set enrichment analysis using a hypergeometric test (Fisher's exact test). The Gene Ontology Biological Process, Reactome and WikiPathways sources are going to be used as pathway databases. We will also upload our custom geneset file.

g:Profiler can be used using the website at <https://biit.cs.ut.ee/gprofiler/gost>. However, for this practical lab, we will run it from the g:Profiler R package.

We will run the query and explore the table of results and visualize the results as bar and dot plots.

One of the greatest features of g:Profiler is that it is updated on a regular basis and most of the previous versions are available online on the gprofiler archive.

The `gprofiler2` -g:Profiler R implementation is a wrapper for the web version. You require an internet connection to get enrichment results.

0.11 Data

`g:Profiler` requires a list of genes. For this analysis, we used genes we found to be differentially expressed in palbociclib-resistant samples compared with parental samples from the breast cancer cell line MCF7 (see chapter 1).

0.12 Exercise 1 - prepare the gene lists

This exercise requires the file “`edgeR_resistant_vs_parental_all_genes.csv`” located in the results folder.

0.12.1 Step 1 - Install and Load libraries

```
# CRAN and Bioconductor setup
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
# List of CRAN packages
cran_packages <- c(
  "tidyverse",
  "knitr",
  "kableExtra",
  "glue",
  "RCurl",
  "webshot2",
  "ggridges",
  "igraph",
  "stringr",
  "data.table",
  "patchwork",
  "ggdendro"
)
# List of Bioconductor packages
bioc_packages <- c(
  "gprofiler2",
  "GSA",
  "fgsea",
  "clusterProfiler",
  "enrichplot"
)
# Install CRAN packages if not already installed
for (pkg in cran_packages) {
```

```

  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
}
# Install Bioconductor packages if not already installed
for (pkg in bioc_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    BiocManager::install(pkg)
  }
}
# Load the libraries
library(tidyverse)
library(knitr)
library(kableExtra)
library(glue)
library(RCurl)
library(webshot2)
library(ggribes)
library(igraph)
library(stringr)
library(data.table)
library(patchwork)
library(ggdendro)

# Bioconductor packages
library(gprofiler2)
library(GSA)
library(fgsea)
library(clusterProfiler)
library(enrichplot)

```

0.12.2 Step 2 - Prepare the define gene lists

```

res = read.csv("results/edgeR_resistant_vs_parental_all_genes.csv")

UPgenes <- res$gene[res$logFC>0 & res$FDR <= 0.001]
DOWNgenes <- res$gene[res$logFC<0 & res$FDR <= 0.001]

```

0.12.3 Step 3 - Run g:Profiler

The next step is to run pathway enrichment analysis using `gost` g:Profiler using the `g:Profiler2` R package. For detailed descriptions of all the parameters that can be specified for the `gost` g:profiler function see the R package information - [here](#) and [here](#).

For this query we are specifying -

- query - the set of genes of interest, as loaded in from the gene list file (`query_set`).
- significant - set to FALSE because we want g:Profiler to return all the results not just the ones that it deems significant by its predetermined threshold. We will filter the results for significance in a next step.
- ordered_query - set to FALSE (because we are not taking into account the order of the list)
- exclude_iea - set to TRUE. We are removing the electronic inferred annotations from the GO database
- correction_method - set to fdr. by default g:Profiler uses g:Scs
- organism - set to "hsapiens" for homo sapiens. Organism names are constructed by concatenating the first letter of the name and the family name (according to gprofiler2 documentation)
- source - the geneset source databases to use for the analysis. We recommend using GO biological process (GO:BP), WikiPathways (WP) and Reactome (Reac) but there are additional sources you can add (GO molecular function or cellular component(GO:MF, GO:CC), KEGG, transcription factors (TF), microRNA targets (MIRNA), corum complexes (CORUM), Human protein atlas (HPA), Human phenotype ontology (HP))

0.12.3.1 run g:Profiler with the up genes

```
query_set = UPgenes

gprofiler_results_UPgenes <- gost(query = query_set ,
  significant=TRUE,
  ordered_query = FALSE,
  exclude_iea=TRUE,
  correction_method = "fdr",
  organism = "hsapiens",
  source = c("REAC", "WP", "GO:BP"))
```

0.12.3.2 observe the results

```
#get the gprofiler results table
enrichment_results <- gprofiler_results_UPgenes$result

#display the names of all columns
colnames(enrichment_results)
```

## [1]	"query"	"significant"	"p_value"
## [4]	"term_size"	"query_size"	"intersection_size"
## [7]	"precision"	"recall"	"term_id"
## [10]	"source"	"term_name"	"effective_domain_size"

```
## [13] "source_order"          "parents"
#display the top rows
head(enrichment_results)

##      query significant      p_value term_size query_size intersection_size
## 1 query_1          TRUE 0.005977645      635         74             13
## 2 query_1          TRUE 0.005977645      509         74             11
## 3 query_1          TRUE 0.005977645       61         74             5
## 4 query_1          TRUE 0.005977645     1013         74             16
## 5 query_1          TRUE 0.005977645      188         74             7
## 6 query_1          TRUE 0.005977645       73         74             5
##      precision      recall      term_id source
## 1 0.17567568 0.02047244 GO:0035295 GO:BP
## 2 0.14864865 0.02161100 GO:0001944 GO:BP
## 3 0.06756757 0.08196721 GO:0050818 GO:BP
## 4 0.21621622 0.01579467 GO:2000026 GO:BP
## 5 0.09459459 0.03723404 GO:0050817 GO:BP
## 6 0.06756757 0.06849315 GO:0032963 GO:BP
##                                     term_name effective_domain_size
## 1                                     tube development             16180
## 2                                     vasculature development             16180
## 3                                     regulation of coagulation             16180
## 4 regulation of multicellular organismal development             16180
## 5                                     coagulation             16180
## 6                                     collagen metabolic process             16180
##      source_order      parents
## 1          8374 GO:00072....
## 2           542 GO:00487....
## 3         12418 GO:00508....
## 4         23538 GO:00072....
## 5         12417 GO:0032501
## 6          7434 GO:0008152
```

0.12.3.3 YOUR TURN: run g:Profiler with the down genes (use same parameters as above)

```
##Tip: remove eval=FALSE chunk option to run the code
query_set = XXXX

gprofiler_results_DOWNgenes <- gost(query = XXXX,
                                     significant = XXXX,
                                     ordered_query = XXXX,
                                     exclude_iea = XXXX,
                                     correction_method = XXXX,
                                     organism = XXXX,
```

```
source = c("XXXX", "XXXX", "XXXX"))

##observe the results (same as for the UP genes)
```

0.12.4 Clear visualization of the results

The `enrichment_results` dataframe contains different columns. Let's rearrange the table to display the most important information. We will arrange the table so it contains: the database origin, the name of each of the pathway, term size, query size, intersection size and adjusted pvalue. -

- Term size: number of genes in the pathway as it is in the database
- Query size: number of genes in our gene list
- Intersection size: number of genes overlapping between our gene list and the tested pathway
- pvalue: adjusted pvalue (corrected for multiple hypothesis testing using the Benjamini-Hochberg method)

The results are ordered by significance, from the most significant pathway (lowest p-value) to the least significant.

```
enrichment_results <- gprofiler_results_UPgenes$result
enrichment_results$p_value <- formatC(enrichment_results$p_value, format = "e", digits

enrichment_results = enrichment_results[ , c("source", "term_name", "term_size", "query

kable(head(enrichment_results, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.4.1 YOUR TURN: visualize the results with the down genes (use same parameters as above)

```
##Tip: remove eval=FALSE chunk option to run the code

enrichment_results <- XXXX
enrichment_results$p_value <- formatC(XXXX)

enrichment_results = enrichment_results[ , c("XXXX", "XXXX", "XXXX", "XXXX", "XXXX", "X

kable(head(enrichment_results, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.5 Step 4 - Explore top table of results

For this analysis, we chose to use 3 databases: GO:BP, Reactome and Wikipathways. If we look at the top table only, we are only going to see the results

Table 1: Enrichment Result

source	term_name	term_size	query_size	intersection_size	p_value
GO:BP	tube development	635	74	13	5.98e-05
GO:BP	vasculature development	509	74	11	5.98e-05
GO:BP	regulation of coagulation	61	74	5	5.98e-05
GO:BP	regulation of multicellular organismal development	1013	74	16	5.98e-05
GO:BP	coagulation	188	74	7	5.98e-05
GO:BP	collagen metabolic process	73	74	5	5.98e-05
GO:BP	anatomical structure morphogenesis	1847	74	22	5.98e-05
GO:BP	cell adhesion mediated by integrin	80	74	5	7.04e-05
GO:BP	blood vessel morphogenesis	457	74	10	8.11e-05
GO:BP	tube morphogenesis	562	74	11	8.11e-05
GO:BP	regulation of multicellular organismal process	2293	74	24	8.15e-05
GO:BP	blood vessel development	487	74	10	1.04e-04
GO:BP	positive regulation of blood coagulation	21	74	3	1.44e-04
GO:BP	positive regulation of hemostasis	21	74	3	1.44e-04
GO:BP	circulatory system development	745	74	12	1.45e-04
GO:BP	regulation of blood coagulation	57	74	4	1.45e-04
GO:BP	regulation of hemostasis	58	74	4	1.46e-04
GO:BP	wound healing	340	74	8	1.46e-04
GO:BP	positive regulation of coagulation	23	74	3	1.46e-04
GO:BP	blood coagulation	182	74	6	1.61e-04

Table 2: Enrichment Result

query	significant	p_value	term_size	query_size	intersection_size	precision	re
query_1	TRUE	0.0000077	298	57	12	0.2105263	0.0402
query_1	TRUE	0.0008208	89	57	6	0.1052632	0.0674
query_1	TRUE	0.0021054	67	57	5	0.0877193	0.0746
query_1	TRUE	0.0029160	76	57	5	0.0877193	0.0657
query_1	TRUE	0.0033756	44	57	4	0.0701754	0.0909
query_1	TRUE	0.0033756	84	57	5	0.0877193	0.0595
query_1	TRUE	0.0097759	60	57	4	0.0701754	0.0666
query_1	TRUE	0.0256174	140	57	5	0.0877193	0.0357
query_1	TRUE	0.0316913	39	57	3	0.0526316	0.0769
query_1	TRUE	0.0322168	42	57	3	0.0526316	0.0714
query_1	TRUE	0.0322168	516	57	9	0.1578947	0.0174
query_1	TRUE	0.0338224	44	57	3	0.0526316	0.0681
query_1	TRUE	0.0449470	675	57	10	0.1754386	0.0148

of GO:BP but it is interesting to look at the top results for each of the 3
databunique(enrichment_results\$source)

```
enrichment_results <- gprofiler_results_UPgenes$result
##Get the name of the databases
unique(enrichment_results$source)

## [1] "GO:BP" "REAC" "WP"

##Filter the dataframe to retrieve only the Reactome results
enrichment_results_Reac <- enrichment_results %>%
  filter(source == "REAC")

##Now display the top 20 pathways
kable(head(enrichment_results_Reac, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.5.1 YOUR TURN: DISPLAY THE TOP 20 PATHWAYS FOR THE WIKIPATHWAY DATABASE (“WP”) and the down genes

```
enrichment_results <- gprofiler_results_DOWNgenes$result
##Get the name of the databases
unique(enrichment_results$source)

##Filter the dataframe to retrieve only the Reactome results
enrichment_results_WP <- enrichment_results %>%
  XXXX(source == "XXXX")
```

```
##Now display the top 20 pathways
kable(head(enrichment_results_WP, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.6 Step 5 - Filter results by geneset size

Restrict the results to just the ones that have at maximum gene-set size of 1000 and a minimum gene-set size of 10 and with an adjusted pvalue of 0.05.-

- maximum gene-set size of 1000: will remove very generic pathway terms that are less informative like “regulation of gene expression”
- minimum gene-set size of 10: will remove small pathways where the pvalue is significant although the overlap size is very small
- p_thres: aim is to retrieve only pathways that are significant under this adjusted pvalue threshold of 0.01 (1% false positive)

```
enrichment_results <- gprofiler_results_UPgenes$result

min_gs_size = 10
max_gs_size = 1000
p_thres = 0.01
# filter by params defined above
# by default we have set the max and min gs size to 250 and 3, respectively.

##the pvalue was stored as a character vector, we have to transform back into a numerical vector
enrichment_results$p_value <- as.numeric(enrichment_results$p_value)

enrichment_results_mxgssize_1000_min_10_adj_0_01 <-
  subset(enrichment_results,
         term_size >= min_gs_size &
         term_size <= max_gs_size &
         p_value <= p_thres )

#
myrows = nrow(enrichment_results_mxgssize_1000_min_10_adj_0_01)

print(glue("It results in {myrows} selected pathways with maximum gene-set size of {max_gs_size}")

## It results in 15 selected pathways with maximum gene-set size of 1000 and minimum gene-set of
```

0.12.6.1 YOUR TURN: using the enrichment results for the down genes, use gene-set size filters and FDR filters and display the results

In a second step, try different thresholds of maximum gene-set size and adjusted pvalue threshold: 500 and 250 for * max_gs_size and 0.05 or 0.001 for pvalue-

- Option 1
 - max_gs_size = 500
 - p_thres = 0.001

or

- Option 2
 - max_gs_size = 250
 - p_thres = 0.05

How many pathways do you get for each option (nrow())??

```
enrichment_results <- gprofiler_results_DOWNgenes$result

min_gs_size = XXXX
max_gs_size = XXXX
p_thres = XXXX
# filter by params defined above
# by default we have set the max and min gs size to 250 and 3, respectively.

##the pvalue was stored as a character vector, we have to transform back into a numeric
enrichment_results$p_value <- as.numeric(enrichment_results$p_value)

enrichment_results <- XXXXX

#
myrows = nrow(enrichment_results)

print(glue("It results in {myrows} selected pathways with maximum gene-set size of {max_gs_size} and p-value threshold of {p_thres}"))
```

0.12.7 Step 6 - Run g:profiler with your own genesets (example using BaderLab genesets)

With regards to pathway sets there are two options when using g:Profiler -

- Use the genesets that are supplied by g:Profiler as we just did.
- Upload your own genesets.

The most common reasons for supplying your own genesets is the ability to use up to date annotations or in-house annotations that might not be available in the public sphere yet. It can be used to test the enrichment in a particular

pathway of interest that might not be in the general pathway database but that you obtained for example by extracting the data from a published paper. You need to format this pathway in a .gmt file.

In this example, we will upload the Baderlab gene set file that contains multiple database sources. We filtered the gene-set prior to uploading the gene-set file to g:Profiler. The Baderlab gene set file combines different sources of databases and you can find more information at: <https://baderlab.org/GeneSets>

''Why do we filter the geneset before running gprofiler''

The g:Profiler interface only allows for filtering genesets by size only after the analysis is complete. After the analysis is complete means the filtering is happening after Multiple hypothesis testing. Filtering prior to the analysis will generate more robust results because we exclude the uninformative large genesets prior to testing changing the sets that multiple hypothesis filtering will get rid of.

We have 1 example of gmt files with different filtering thresholds : * genesets less of equal than 500 genes

Load in the GMT file

This step will take the gmt file that we downloaded locally and upload them into our R environment in the GSA.genesets object called "genesets_baderlab_genesets". The capture.output() function is only there to redirect the printed messages for a clearer notebook.

The format of the GMT file is described https://software.broadinstitute.org/cancer/software/gsea/wiki/index.php/Data_formats and consists of rows with the following

- Name
- Description
- tab delimited list of genes a part of this geneset

Write out the gmt file with genenames

In order to use your own genesets with g:Profiler you need to upload the file to g:Profiler server first. The function will return an ID that you need to specify in the organism parameter of the g:Profiler gost function call. It is done by using the upload_GMT_file() function which is included in the gprofiler2 package. Note: it took about 2 minutes to run this chunk of code.

```
dest_gmt_file_500 ="data/Human_GOBP_AllPathways_filtered_500orLessV2.gmt"

custom_gmt_max500 <- upload_GMT_file(
  gmtfile=dest_gmt_file_500)
```

Your custom annotations ID is gp__ZMMW_rs4V_Duc.

You can use this ID as an 'organism' name in all the related enrichment tests against this cus

Just use: gost(my_genes, organism = 'gp__ZMMW_rs4V_Duc')

```

UPgenes <- res$gene[res$logFC>0 & res$FDR <= 0.001]
DOWNgenes <- res$gene[res$logFC<0 & res$FDR <= 0.001]

gprofiler_results_custom_max500_UPgenes <- gost(query = UPgenes ,
  significant=TRUE,
  ordered_query = FALSE,
  exclude_iea=TRUE,
  correction_method = "fdr",
  organism = custom_gmt_max500
)

## Detected custom GMT source request
enrichment_results_UPgenes <- gprofiler_results_custom_max500_UPgenes$result

enrichment_results_UPgenes$parents <- sapply(
  enrichment_results_UPgenes$parents,
  function(x) paste(x, collapse = ", ")
)

write.csv( enrichment_results_UPgenes, "results/enrichment_results_gprofiler_UPgenes.csv" )

```

0.12.7.1 YOUR TURN: perform the same task with the down genes

```

gprofiler_results_custom_max500_DNgenes <- gost(
  query = XXXX ,
  significant=XXXX,
  ordered_query = XXXX,
  exclude_iea=XXXX,
  correction_method = "XXX",
  organism = XXXX
)

enrichment_results_DNgenes <- gprofiler_results_custom_max500_DNgenes$result

enrichment_results_DNgenes$parents <- sapply(
  enrichment_results_DNgenes$parents,
  function(x) paste(x, collapse = ", ")
)

write.csv( enrichment_results_DNgenes, "results/enrichment_results_gprofiler_DNgenes.csv" )

```

0.13 Exercise 2 - visualization

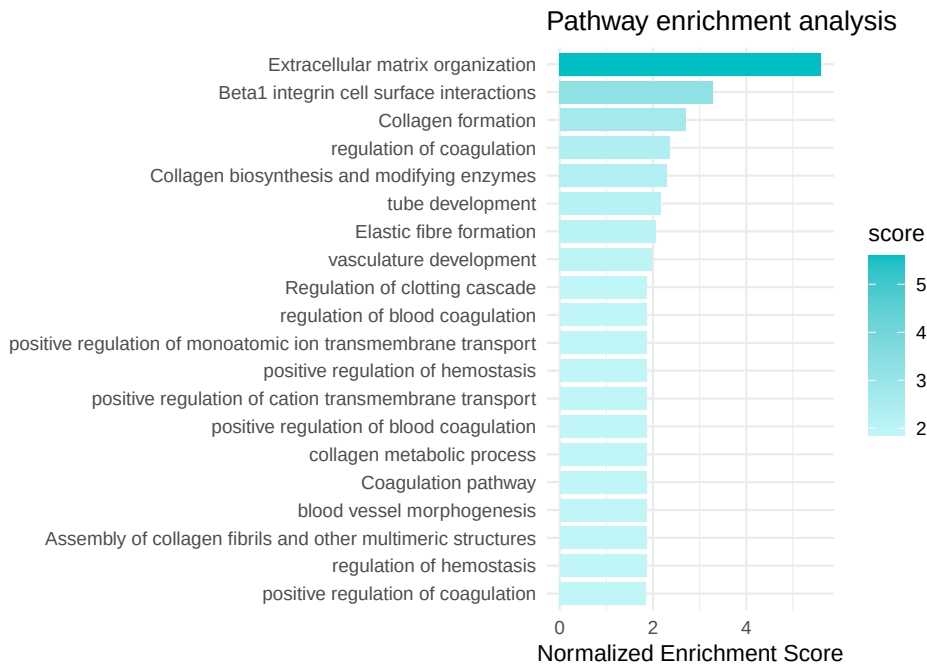
We will use clusterProfiler style visualization. *### Step 1 - Bar plot*

##Calculate a score (small very significant adjusted p value will turn into a large score)
`enrichment_results_UPgenes$score = -log10(as.numeric(enrichment_results_UPgenes$p_value))`

```
enrichment <- enrichment_results_UPgenes %>%
  as_tibble() %>%
  arrange(desc(score)) %>%
  slice_head(n = 20)

p = ggplot(enrichment, aes(reorder(term_name, score), score)) +
  geom_col(aes(fill = score), width=0.8) +
  scale_fill_gradient(low = "#C1F6F8", high = "#00BFC4")+
  coord_flip() +
  labs(x="", y="Normalized Enrichment Score",title="Pathway enrichment analysis ") +
  theme_minimal()
```

p



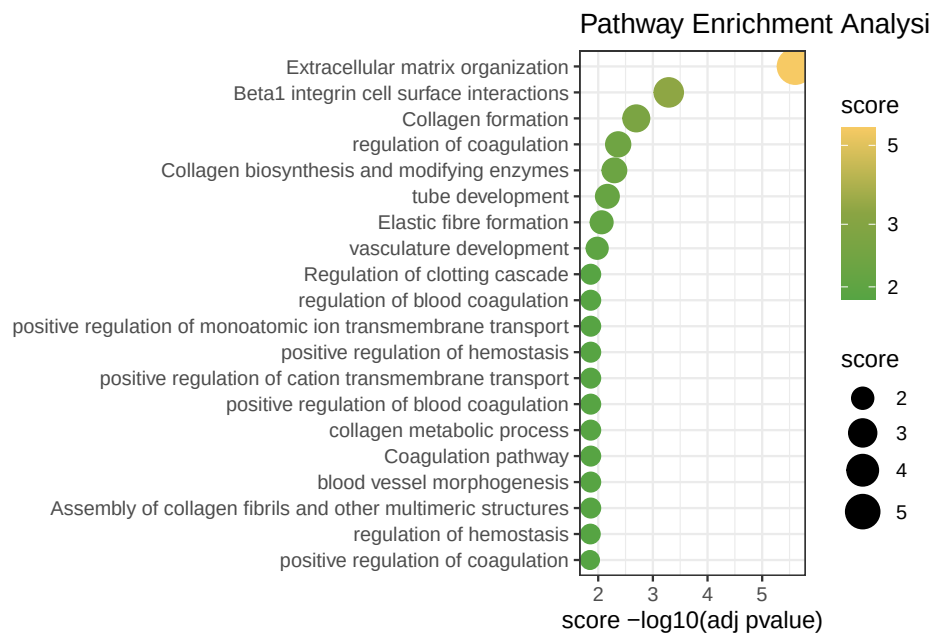
0.13.1 Step 2 - Dot plot with segment

```

enrichmentTop20 <- enrichment %>%
  arrange(desc(score)) %>%
  slice_head(n = 20)

p = ggplot(enrichmentTop20, aes(
  y = fct_reorder(term_name, score),
  x = score,
  color = score,
  size = score
)) +
  geom_point() +
  scale_color_gradientn(
    colours = c("#56A443", "#8AA443", "#f7ca64"),
    trans = "log10",
    guide = guide_colorbar(reverse = FALSE, order = 1)
  ) +
  scale_size_continuous(range = c(4, 8)) +
  theme_bw(base_size = 12) +
  xlab("score -log10(adj pvalue)") +
  ylab(NULL) +
  ggtitle("Pathway Enrichment Analysis") +
  theme(
    axis.text.y = element_text(size = 10)
  )
p

```



Ranked gene list

You can add parts to organize one or more book chapters together. Parts can be inserted at the top of an .Rmd file, before the first-level chapter heading in that same file.

Add a numbered part: `# (PART) Act one {-}` (followed by `# A chapter`)

Add an unnumbered part: `# (PART\*) Act one {-}` (followed by `# A chapter`)

Add an appendix as a special kind of un-numbered part: `# (APPENDIX) Other stuff {-}` (followed by `# A chapter`). Chapters in an appendix are prepended with letters instead of numbers.

Dataset is coming from : <https://pubmed.ncbi.nlm.nih.gov/37384539/>

Pathway Enrichment Analysis reference paper: